

Article

Is Code Co-Committal an Indicator of Evolutionary Coupling in Software Repositories?

Niall Price¹, David Cutting^{1,*}  and Vahid Garousi^{1,2} 

¹ School of Electronics, Electrical Engineering and Computer Science, Queen's University Belfast, Belfast BT7 1NN, UK; v.garousi@qub.ac.uk (V.G.)

² Institute of Data Science and Artificial Intelligence, Azerbaijan Technical University (AzTU), Baku AZ 1073, Azerbaijan

* Correspondence: d.cutting@qub.ac.uk; Tel.: +44-(0)28-9097-4998

Abstract

Software repositories such as Git are significant sources of metadata about software projects, containing information such as modified files, change authors, and often commentary describing the change. An emerging approach to support software change impact analysis is to exploit this metadata to determine which files are linked by co-committal, i.e., when two files are frequently updated together within the same Git commit. Such information can serve as an indicator for identifying potential change-impact sets in future development activities. The aim of this study is to determine whether co-committal is a reliable indicator of links between software artifacts stored in Git and, if so, whether these links persist as the artifacts evolve—thereby offering a potentially valuable dimension for change impact analysis. To investigate this, we mined the metadata of five large Git repositories comprising over 14K commits and extracted co-change sets from the resulting data. The results show that: (1) co-committal links between artifacts vary widely in both strength and frequency, with these variations strongly influenced by the development style and activity levels of the contributing developers, and (2) although co-committal can serve as an indicator of evolutionary coupling in certain scenarios, its usefulness depends on project-specific development practices and observable patterns of developer behavior.

Keywords: software evolution; mining software repositories; change impact analysis; change coupling; co-committal

1. Introduction

Making changes to software systems inherently carries a risk that the changes made will have wider-reaching and/or unwanted impacts on the system than previously anticipated. There are numerous proposed approaches and methodologies used to attempt to identify possible impacts of change in order to mitigate this risk [1,2]. These approaches include static and dynamic code analysis as well as the generation of change-impact sets using co-committal data.

It has been debated whether co-committal (also referred to as “co-change” or “joint commits”) patterns provide a useful dimension for use in change-impact analysis, e.g., if changing a single artifact within an established co-committal pattern will have an effect on other artifacts within said pattern. Furthermore, the literature has questioned not only the usefulness of co-committal relationships in change impact analysis but also whether their usability can be sustained throughout the evolution of their respective software systems for



Academic Editor: Ian Gorton

Received: 29 December 2025

Revised: 15 February 2026

Accepted: 26 February 2026

Published: 1 March 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

use in long-term analysis, an issue known as evolutionary coupling (also known as “logical coupling”) [3,4]. In this study, evolutionary coupling is defined as the degree to which software artifacts exhibit interdependent change over time, reflecting shared evolution driven by functional, architectural, or process-related dependencies.

A key distinction of this study is its reliance on serialized commit data, in which commits are analyzed in their natural temporal order and author context rather than as aggregated co-change events. Unlike traditional MSR approaches that collapse commit histories into unordered co-change counts or static graphs, serialized commit data preserves when, how often, and by whom files are co-modified. This enables analysis of whether co-committal relationships persist across time and contributors or arise only from isolated development episodes, allowing incidental co-changes to be distinguished from sustained evolutionary coupling. Prior co-change studies typically analyze aggregated commit histories, treating commits as unordered events. In contrast, we analyze co-committal within serialized commit data, preserving commit order and developer context, which enables us to distinguish persistent co-evolution from isolated or incidental co-changes.

As a conceptual property of artifact co-evolution, evolutionary coupling is not inherently tied to a specific measurement technique; in this work, it is operationalized through co-committal, an observable proxy derived from developers’ commit behavior. Co-committal is quantified using two complementary metrics: Co-Committal Strength (CCS), which captures the relative frequency with which a pair of artifacts is modified together across commits, and Co-Committal Frequency (CCF), which captures the absolute number of commits in which the same artifact pair is co-modified. Together, these metrics characterize both the intensity and recurrence of co-committal relationships.

The aim of this work is to answer the following Research Question (RQ): To what extent is evolutionary coupling a common trait exhibited by software artifacts that are frequently co-committed using Git?

A further aim is to use the analysis of the RQ to explore to what extent co-committal may be a useful dimension for software change-impact analysis, an often cited potential use for these relationships. The adopted approach to answer RQ1 is to extract and analyze the change sets of selected open-source repositories against defined metrics (see Section 3.2). The novelty of this approach comes in part from the dataset, as no preexisting work has considered using serialized commit data to generate results. The approach will take into consideration any additional dimensions raised by existing research to refine results, such as commit content and frequency, as well as the behavioral patterns of commit authors.

We implemented an activity threshold to examine whether co-committal patterns correlate with developer activity, and we varied the threshold across multiple levels to assess its effect on observed coupling. Additionally, we analyze the scope of limiting the dataset to only repositories with below a certain number of authors, to determine whether projects with fewer developers have more consistent co-committal patterns.

It was expected that different repositories would vary in the co-committal strengths found, and that changing the thresholds on author activity and minimum co-committal strength of links will have an impact on the predictability and strength of the average co-committal. It was also expected that the behavioral patterns observed in the dataset will vary depending on the development style of the authors within each repository.

2. Related Work

The most common approach which was found in existing literature to explore co-committal patterns was the use of co-change data in order to create connected component graphs and thus perform social network analysis (SNA) on software artifacts [5,6]. This is a form of analysis where a network graph is created with software artifacts as nodes and the

relationships, instances of co-committal, as the edges to visualize connections. The merit of this approach is that it is “capable of revealing system characteristics that are particularly useful for the development team” and also provides insights for quantitative metrics such as change set centrality and community structure [7]. Additionally, ref. [8] found that SNA measures are more accurate than traditional analysis in predicting software faults which were caused by evolving dependencies. However, artifacts in such a graph “will have to be weighted appropriately in order to represent how frequently they change with each other” [8].

Cohen et al. [6] used the top 1000 GitHub public repositories for each of 17 programming languages as their dataset in order to analyze co-committal patterns on a large scale. Further, they narrowed down the data to developers who contributed above 75% of the average number of commits per developer per repository using an “activity threshold”. The authors acknowledged that results were highly dependent on the value of the activity threshold, along with the fact that they only considered the frequency of commits rather than their semantic content. This presents identifiable gaps for further research.

Commit content is an area discussed by several papers, which found that bug fix-related commits had more value to change impact analysis than those which added new features, as well as that large numbers of small commits provided more accurate results as opposed to “tangled commits” (where one commit contains a large number of artifacts) [9,10]. An overabundance of tangled commits could potentially lead to categorization of commits as being primarily composed of bug fixes [11]. It was also found that “adding a new method declaration, and changing the method body have shown a negative correlation with the caused impact, while the change type changing field type has shown a positive correlation”, meaning that commits with these specific features were more commonly linked to predictable change impacts [12]. This implies that the utility of the results of some methodologies for co-committal analysis are highly dependent on the choice of repository.

Another observation made in existing literature on the importance of building an appropriate dataset comes from [13], in which it was found that “all the transactions are not equivalent in terms of prediction behavior” and that involving the age and history length of the commits being analyzed in their prediction algorithm led to an improvement in prediction quality. This highlights the importance of considering the external metadata elements of commits—if a co-commit was made in the past and there have been many more commits since then, it may not be as useful for change impact analysis as the dependencies may have evolved past this commit.

Hanam et al. [14] raised the possibility of noise in change impact analysis data. Their changes in order to combat this within their study were to calculate the difference in abstract syntax trees between program versions in order to reduce false positives and negatives, demonstrating the significance of considering noise reduction methods in this project, for which aforementioned features such as activity thresholds and semantic filters may also be useful.

Huang et al. [15] found that they could achieve more accurate change impact analysis results by mapping change patterns to equivalent classes from historical change sets. This could positively support the hypothesis that evolutionary coupling is an accurate dimension for change impact analysis.

Ramadani and Wagner [16] used commit analysis tools to collect commit logs, and then used a data mining framework to generate co-change data. Their research was focused on the effect of branching and time between commits on the relatedness of file change sets; they note that some of the most strongly linked change sets were committed in a single day, and that branching had minimal impact on relatedness, but that these results were dependent on the developer. This paper had less focus on software evolution, which would indicate a gap in their research that this project could aim to fill.

Higo et al. [17] used an MSR tool called Hstorage as well as tools used in [16] to analyze repositories and identify evolutionary coupling. They focus more on the code itself rather than its repository metadata, including treating Java methods as files and focusing on a higher level of granularity in their analysis of Git metadata. They also use git log to check data, including using it to count the number of authors who committed a file, but added a stipulation that they would track files across commits by identifying text-based similarity in the change logs and using it to equate files through semantic links. They also use a tool called FinerGit, which they found identified evolutionary coupling with more accuracy than other explored alternative tools.

Mondal et al. [18] reported an empirical study on bug-fix commits, including implementing a tool to detect them. This is because they found methods which experience bug-fix commits have a higher level of evolutionary coupling. This would be enforced by the previously-made suggestion that lower CCS, i.e., higher proportion of internal changes to a file, indicates higher CCF with others, i.e., longer evolutionary coupling; if a file changes more by itself, it will also change more with others. They identify the percentage of bug-fix commits and correlate it with the percentage of methods which are coupled by co-committal and found that there was a high positive correlation between them, meaning that the more bug-prone methods are, the more likely they are to be evolutionarily coupled which has implications as to the applicability of the approach to change impact. They also acknowledged that their detection of both metrics may vary in accuracy, which would be a minor threat to the validity of their results.

Saydemir et al. [19] discusses the use of software architecture recovery tools which cluster together records from a dependency graph to identify dependencies, i.e., semantic links, between modules in each repository. They found that considering evolutionary coupling increased the accuracy of their results by a significant amount. They take a small number of open-source repositories and use a tool to parse their metadata and identify links, which is a similar methodology to what has been proposed earlier in this paper. They also set a threshold on the data based on CCF values (which they referred to as “number of co-changes”) in order to give more focus to detecting potential evolutionary coupling than gathering general data on both CCF and CCS metrics. Their main conclusion was that high similarity generally indicates fewer co-changes, but results were highly dependent on the choice of repository.

Rolfsnes et al. [3] focused on the role of evolutionary coupling in change impact analysis. They developed a tool named TARMAQ which works by evaluating repositories to identify evolutionarily coupled files. After utilizing this tool to analyze repositories containing code from various different languages, they conclude by stating that evolutionary coupling is a promising direction to address the caveats of static and dynamic dependency analysis, with language-agnostic merits.

Sohn and Papadakis [20] discussed use of evolutionary coupling to achieve higher test coverage, applying a specific development-focused use case to their research. Their research is motivated by a noteworthy hypothesis: “Changes in code units that are followed by the changes in tests (and vice versa) imply a relevance relationship (coupling) between them. Similarly, changes in code units not followed by changes in tests imply an absence of relevance”. This is a very similar concept to the research question of this paper, as they both suggest that direct semantic links between files are implied by co-committal.

Research Gap and Positioning

Prior studies have investigated co-commit relationships and change-set dependencies in large repositories. For example, Cohen and Consens [6] analyzed large-scale co-commit patterns among active developers, while Ramadani and Wagner [16] examined relation-

ships among change sets in Git repositories. These studies primarily focus on identifying co-occurrence or relatedness patterns, often using single metrics or thresholding strategies. However, they provide limited support for systematically characterizing developer coupling using multiple complementary measures or for quantifying the intensity and consistency of such relationships across activity levels.

In contrast, our work advances the state of the art by combining CCS and CCF thresholds with a multi-metric statistical analysis, enabling finer-grained differentiation between high- and low-activity developers and providing more robust empirical evidence of coupling behaviors. This integrated approach allows us to move beyond simple detection of co-commit events toward a more comprehensive understanding of their strength, distribution, and practical implications.

3. Data Collection, Metrics, and Analysis Approach

3.1. Selected Git Repositories and Dataset

Table 1 summarizes the five repositories that were selected to be analyzed. Our selection criteria included variations in project sizes, number of commits and developers. The repositories were selected to span a range of project sizes and activity levels, allowing comparison between highly active, large-scale projects and smaller, less active ones in terms of both number of commits and contributing developers. This variation enables exploration of how co-committal behavior and evolutionary coupling manifest under different development conditions, rather than assuming uniform project dynamics.

The analysis reported in this paper is based on commit-level change information extracted from the selected Git repositories. For each repository, we analyzed all commits available up to March 2024. The extracted data included commit identifiers, timestamps, authorship information, and the set of files modified in each commit. Source-code diffs or semantic code changes were not analyzed; instead, changes are represented at the level of file co-modification within commits.

Data were collected in March 2024. The repositories contained 14,008 commits in total.

Table 1. Overview of the Analyzed Repositories.

Project Name	# Commits	# Dev's	URL (Accessed on 8 January 2024)
1-Cal.com	8687	514	https://github.com/calcom/cal.com
2-Clash Verge Rev	1303	41	https://github.com/clash-verge-rev/clash-verge-rev
3-Google Maps Services Client	1226	106	https://github.com/googlemaps/google-maps-services-java
4-Sway	2591	109	https://github.com/FuelLabs/sway
5-OLMo-Eval	161	5	https://github.com/allenai/OLMo-Eval

3.2. Data Collection and Tool Support

The Git repositories taken from GitHub had their metadata formatted using the `git log` command before being converted to JSON using a publicly available tool (CLOSER [21]). The JSON was then imported into a relational database to allow it to be queried to produce summary metadata about each repository in the dataset.

Data extraction and pre-processing were performed using the CLOSER framework. We configured the CLOSER tool to mine commit histories directly from Git repositories. The framework stored extracted commit metadata in a relational database, where commits, authors, files, and co-committal relationships are represented as separate entities. This organization enabled efficient querying of co-committal strength and frequency under different threshold configurations.

The summary data included the following: mean co-committal strength and frequency per link, the mean number of commits per day, the mean number of commits per author, and the mean number of file changes per commit.

3.3. Metrics

For the purpose of this study, we defined and used the following metrics.

3.3.1. Co-Committal Frequency (CCF) and Co-Committal Strength (CCS)

The Co-Committal Frequency (CCF) and Co-Committal Strength (CCS) metrics are defined as the number of co-commits involving each artifact both as an integer and as a percentage of the overall number of times the artifact was committed. For example, if every time artifact *A* was committed, it was co-committed with artifact *B*, then the co-committal link *AB* has a CCS of 100%. As another example, suppose artifact *A* is committed 4 times overall, and in 2 of those commits it is co-committed with artifact *B*. In this case, $CCS_{A \rightarrow B} = 50\%$, and the corresponding CCF value is $CCF_{A,B} = 2$.

It is important to clarify that Co-Committal Strength (CCS) is a directional relationship. For two artifacts *A* and *B*, $CCS_{A \rightarrow B}$ measures the percentage of commits containing *A* in which *B* also appears, whereas $CCS_{B \rightarrow A}$ measures the percentage of commits containing *B* in which *A* also appears. These values generally differ because *A* and *B* may have different total numbers of commits. For example, if *A* is modified many times independently while *B* is rarely modified without *A*, then $CCS_{A \rightarrow B}$ will be low while $CCS_{B \rightarrow A}$ will be high. In contrast, Co-Committal Frequency (CCF) is non-directional because it counts the absolute number of commits in which *A* and *B* appear together.

3.3.2. Illustrative Example of CCS and CCF

To clarify the distinction between $CCS_{A \rightarrow B}$ and $CCS_{B \rightarrow A}$, consider the following simplified example involving three files: *A*, *B*, and *C*. Assume the repository contains the following four commits:

- Commit 1: {*A*, *B*}
- Commit 2: {*A*}
- Commit 3: {*A*, *B*, *C*}
- Commit 4: {*A*, *C*}

From these commits, the co-committal frequency (CCF) between *A* and *B* is: $CCF_{A,B} = 2$ (Commits 1 and 3).

We now compute the directional CCS values:

- File *A* appears in 4 commits; *B* appears alongside *A* in 2 of them. Therefore: $CCS_{A \rightarrow B} = 2/4 = 50\%$.
- File *B* appears in only 2 commits, and *A* appears alongside *B* in both. Therefore: $CCS_{B \rightarrow A} = 2/2 = 100\%$.

This example illustrates why $CCS_{A \rightarrow B}$ and $CCS_{B \rightarrow A}$ may differ substantially even though the underlying co-change events (i.e., the CCF) are identical. CCF captures the absolute number of shared commits, whereas CCS quantifies the relative dependency of one file's change history on another's.

3.3.3. Thresholding of Metric Values

To analyze the behavior of file links with higher CCS values, and an activity threshold to analyze the behavior of higher-contributing authors to the repositories, thresholding capabilities were implemented in our analysis framework, as discussed next.

CCS Threshold

In order to identify the proportion of file links in each repository with CCS values above a predetermined level, a CCS threshold was implemented. This threshold allows for the analysis of the behavior of file links at various different levels of CCS, such as evaluating

how link strength affects CCF, the number of links above the threshold per file, and the average CCS of a link.

Activity Threshold

As a further development upon the work done in [6], an activity threshold was implemented in order to analyze whether the activity level of the most active developer had an impact on the presence of co-committal in their work. Given a value between 0 and 1, the activity threshold limits the analysis to authors whose number of commits is at least that fraction of the commit count of the most active author in the repository. An activity threshold of 0 includes all authors in the repository; a threshold of 0.5 includes only authors whose commit count is at least half of that of the most active author in the repository; whereas a threshold of 1 includes only the single most active author.

Analysis was repeated to provide data for these metrics on all repositories within the dataset, with permutations of the activity and CCS threshold parameters ranging from 0 to 1 in increments of 0.2.

Furthermore, the process created for the purpose of analyzing the dataset calculates a set of distinct metrics for each repository in the dataset for the purpose of gathering information on the presence of co-committal and evolutionary coupling. These metrics are as follows:

The activity threshold used in this study differs from percentile-based approaches commonly employed in prior threshold-experiment and sensitivity-analysis work. Instead of selecting developers based on the distribution of commits across all contributors (e.g., top 25% or top 50% by percentile), our definition relates an author's activity to that of the repository's most active contributor. Specifically, an author is included if their commit count is at least a fixed proportion of the commit count of the top contributor. For example, under an activity threshold of 0.6, only authors whose total number of commits is at least 60% of the number of commits made by the top contributor are included. This relative-to-top-contributor scheme provides a clear and reproducible inclusion rule across repositories with widely varying contributor distributions, but it is not identical to percentile-based thresholds used in earlier studies. Future work may explore the relationship between these two approaches and assess whether they produce systematically different results when combined with sensitivity analyses.

Threshold Increment Selection and Sensitivity Considerations

In this study, activity and CCS thresholds were evaluated using increments of 0.2 (i.e., threshold values of 0.0, 0.2, 0.4, 0.6, 0.8, and 1.0). This increment size was selected to balance resolution with computational cost, as each threshold permutation requires recomputing all co-committal metrics across the full dataset. However, we acknowledge that smaller increments (e.g., 0.05 or 0.10) could provide finer-grained insight into threshold effects. A natural extension of this work would be to perform a formal sensitivity analysis, systematically varying thresholds at smaller step sizes to quantify the robustness and stability of observed trends. Such an analysis would help determine whether key patterns reported in this study persist under higher-resolution threshold sweeps.

3.3.4. Total Number of Co-Committal Links

This metric is determined by filtering the dataset to only include commits by authors above the activity threshold, and then counting the number of detected co-committal links between files with a CCS above the threshold. This metric was chosen because it would serve to highlight the overall size of the co-committal link network in each repository, as well as acting as a base for other metrics such as the percentage of co-committal links

above a given threshold. It is expected that as the CCS and activity thresholds increase, this number will decrease as more of the dataset is filtered out of the results, by the threshold.

3.3.5. Percentage of Links Above CCS Threshold Given Activity Threshold

This metric is determined by narrowing down the dataset to only include commits by authors above the activity threshold, counting the number of detected co-committal links between files with a CCS above the threshold, and dividing it by the number of file links evaluated. This metric was chosen in order to provide a perspective on the likelihood of files in each repository having a co-committal link above this threshold. Additionally, it would shed light on any present variation in the proportion of file links above the CCS threshold by authors of various activity levels.

3.3.6. Average # of CCS Links Above Threshold per File

This metric is determined by dividing the number of detected CCS links above the activity and CCS thresholds by the number of files in the repository. This metric was chosen as it can be used to estimate how many files in a repository are linked by co-committal to any given file. It is expected that this value will decrease as the CCS and activity thresholds increase, due to there being fewer file links considered in the evaluation and therefore fewer file links.

3.3.7. Average CCF of Link Above Threshold

This metric is determined by counting the number of times a file was co-committed with another file given that the CCS of the involved file link was above the CCS and activity thresholds, and dividing it by the number of links above the thresholds. This metric was chosen as it is expected to serve as a marker of the level of evolutionary coupling displayed by the average file link.

3.3.8. Average CCS of Link Above Threshold

This is calculated as the total CCS of all links above the thresholds, divided by the number of file links which fit the same criteria. This metric was chosen in order to provide details on the mean level of connectivity in a single co-committal network of a given repository. Additionally, this metric can be compared and contrasted against the average CCF per link metric in order to determine whether files which are co-committed together a higher percentage of the time they are committed correlates to evolutionary coupling.

3.3.9. No. Authors Above Activity Threshold

This metric is determined by counting the number of distinct authors whose number of commits made was higher than the thresholded activity level. It was chosen in order to provide perspective on the size of each repository in the dataset, as well as to serve as a base for other author behavior-based metrics, such as the average number of commits per author above the activity threshold.

3.3.10. Average No. Commits per Author Above Activity Threshold

This is calculated as the total number of commits by an author above the activity threshold, divided by the number of authors above the activity threshold. This metric was chosen in order to serve as an additional dimension to co-committal analysis, as the general behavior of authors whose commits contain higher levels of evolutionary coupling can be determined through comparison between this and other metrics.

3.3.11. Average No. Files Changed per Commit Given Activity Threshold

This is calculated as the total number of files changes made through commits by authors above the activity threshold, divided by the number of authors above the activity threshold. This metric was chosen as it can be used to evaluate the effect of different development styles and activity levels on how “tangled” the average commit is [10], and therefore the likelihood of evolutionary coupling between files co-committed by authors above the activity threshold.

3.3.12. Average Number of Commits per Day

This is calculated as the number of commits made by authors above the activity threshold, divided by the number of distinct dates upon which a commit took place. Given the method in which this metric is calculated, it could also be determined as the expected number of commits made per day above the activity threshold, given that a commit was made on any given day. This metric was chosen as it can provide comparable results between longer and shorter-lived repositories, providing a means of normalization for the data.

It is noteworthy that the average number of commits per day, the number of authors above the activity threshold, the average number of files changed per commits and the average number of commits per author do not vary with the CCS threshold, as their data is solely dependent on the behavior of the author rather than the content of their commits.

For each metric, we report the mean, median, and mode, as well as the standard deviation. This will enable evaluation of the skewness of these results and how much variance there is from an average value, in addition to providing perspective on what the most common value and median percentile value is for each metric.

It is worth comparing the methodology employed and set of metrics recorded in this work to that of the previous literature. The previous works [6,7,10,20] used the median values of the metrics they analyzed, and ref. [18] briefly discussed the use of standard deviation as an analytical metric. Additionally, ref. [16] recorded their mode and standard deviation for time between related commits, but for few other co-committal-related metrics. A study [12] used standard deviation to calculate the Pearson correlation coefficient between bug fix change types and their corresponding levels of change impact. Nonetheless, it can be said that calculating the standard deviation, median and mode in order to evaluate them to the same level of detail as all other metrics in co-committal analysis remains a relatively novel approach and a further development upon previous research into this area.

3.4. Data Analysis Approach

The analyses conducted in this study are exploratory and descriptive in nature. Rather than testing predefined hypotheses, the goal is to characterize patterns of co-committal behavior and to examine how these patterns vary under different activity and co-committal strength (CCS) threshold configurations. To this end, we employ descriptive statistical measures, including mean, median, mode, and standard deviation, to summarize the distributional properties of co-committal strength (CCS), co-committal frequency (CCF), and related commit-level metrics.

The analysis was driven by a set of predefined analytical questions, for which we present the findings in the subsections of the Results Section next, including: how frequently files are co-committed, how strong co-committal relationships are under varying thresholds, how these relationships differ across developer activity levels, and how stable co-committal patterns are across repositories. These questions were operationalized through parameterized queries over the extracted commit metadata rather than through manual inspection of individual changes.

Given the large scale of the dataset, comprising more than 10,000 commits across five repositories, and the systematic replication of analyses across multiple repositories and parameter settings, consistent trends observed in the results are interpreted as meaningful indicators of co-evolutionary behavior. Formal statistical hypothesis testing is not applied, as the study does not aim to make inferential claims or establish causal relationships. Instead, confidence in the findings is derived from the magnitude, consistency, and persistence of observed patterns across repositories, activity thresholds, and CCS thresholds.

The descriptive and exploratory approach allows the study to highlight structural characteristics and behavioral regularities in co-committal relationships while avoiding over-interpretation of statistical variation that may arise from project-specific practices or developer-level differences.

4. Results

This section reports the results of a series of exploratory analyses conducted to address the study's research question: *To what extent is evolutionary coupling a common trait exhibited by software artifacts that are frequently co-committed using Git?* The analyses examine how co-committal relationships vary under different activity and co-committal strength thresholds, and how these relationships manifest across files, developers, and repositories. Unless stated otherwise, results reported in this section refer to aggregated observations across all five repositories. Where notable repository-specific differences occur, they are explicitly identified and discussed to distinguish general trends from project-specific behavior.

4.1. Impact of Activity Threshold on Developer Inclusion

It can be observed in Figure 1 that as the activity threshold increases, the number of authors above the threshold decreases for all repositories, which is an expected outcome and confirms that the threshold effectively reduces the set of included developers.

However, the steep decline observed even at relatively low threshold values reveals a strong concentration of commit activity among a small subset of highly active developers, particularly in larger repositories. This indicates that the majority of contributors participate only sporadically and therefore exhibit limited and infrequent co-committal behavior. As a result, analyses performed at higher activity thresholds focus primarily on a small core of developers whose activity levels are sufficient to produce recurring co-changes between files. This highlights an inherent trade-off between inclusiveness and analytical robustness when using activity-based filtering as a prerequisite for co-committal analysis.

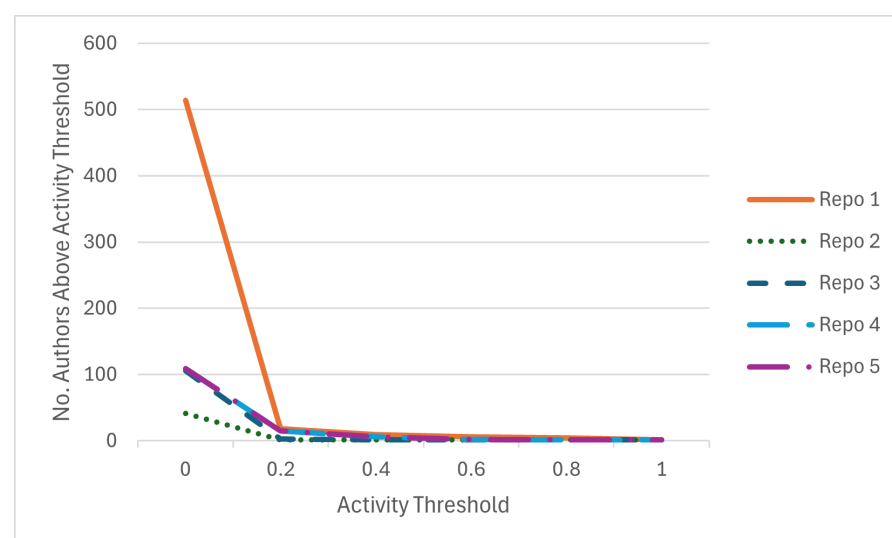


Figure 1. Number of authors for all repositories and activity threshold values from 0 to 1.

4.2. Effect of CCS Threshold Values on Co-Committal Link Formation

As shown in Figure 2, increasing the CCS threshold results in a consistent reduction in both the number and the percentage of co-committal links across all repositories, which confirms the correct implementation of the thresholding mechanism.

The decline is particularly steep at relatively low CCS threshold values, indicating that only a limited proportion of file pairs exhibit sufficiently high co-committal strength to satisfy stricter thresholds. Beyond moderate CCS threshold levels, the proportion of links meeting the threshold rapidly approaches zero for all repositories, demonstrating that most file pairs are characterized by low co-committal strength and are therefore excluded under more restrictive conditions. This finding has direct implications for the feasibility of CCS-based filtering: while higher thresholds isolate file pairs with stronger co-change relationships, they also substantially reduce link coverage, constraining the analysis to a small subset of artifacts. Consequently, practical application of CCS thresholds requires a careful balance between identifying strongly coupled file pairs and maintaining adequate coverage for meaningful analysis.

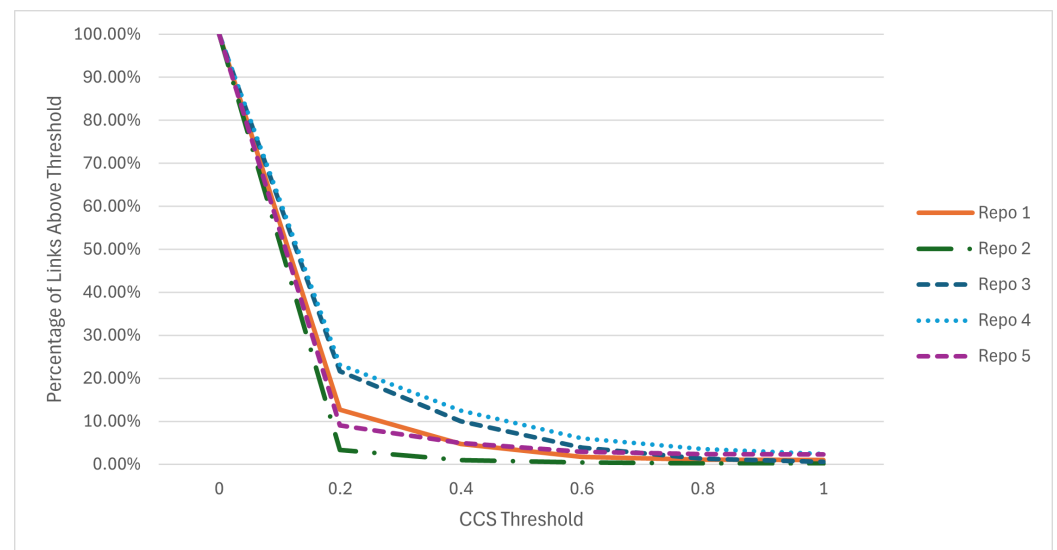


Figure 2. Graph showing percentage of links above CCS threshold for each repository, for CCS threshold values from 0 to 1.

4.3. Effect of Activity and CCS Thresholds on the Share of Strong Co-Committal Links

Figure 3 illustrates the combined effect of activity and CCS thresholding on the proportion of co-committal links retained in the analysis. For all repositories, the percentage of links exceeding a given CCS threshold increases as the activity threshold decreases, even though the absolute number of links is reduced. This indicates that commits authored by more active developers are disproportionately responsible for file pairs that exhibit stronger co-committal behavior, and that filtering out low-activity contributors concentrates the remaining links toward higher CCS values.

The figure further shows that this concentration effect is highly sensitive to the chosen CCS threshold. At low CCS thresholds, a substantial proportion of links are retained across all activity levels; however, as the CCS threshold increases, the percentage of remaining links declines sharply, particularly for lower activity thresholds. This demonstrates that strong co-committal relationships are relatively rare and become increasingly sparse as stricter strength requirements are imposed, regardless of the level of developer activity considered.

Consistent with this trend, the mean, median, and modal number of links per file peak at a CCS threshold of 0%, where all co-committal relationships are included. As the CCS threshold increases, the average CCS of the remaining links approaches 100%, while

the average number of links per file decreases and the corresponding standard deviation increases. This combination indicates that once weak co-committal relationships are filtered out, strong co-committal links are unevenly distributed across files: a small subset of files retains multiple strong links, whereas most files retain few or none.

A particularly notable result is that, across all five repositories, the modal number of links above the threshold per file becomes 0 at CCS thresholds of 40% or higher. This means that for the majority of files, no strong co-committal relationships are observed under stricter threshold settings. Even files that are frequently modified tend to be co-committed with only a limited number of other files, or—especially in the case of files with minimal dependencies—are most often modified in isolation. Taken together, these results highlight a fundamental sparsity in strong co-committal relationships at the file level and indicate that analyses relying on high CCS thresholds inherently focus on a small subset of tightly coupled files, limiting their applicability for comprehensive repository-wide analysis.

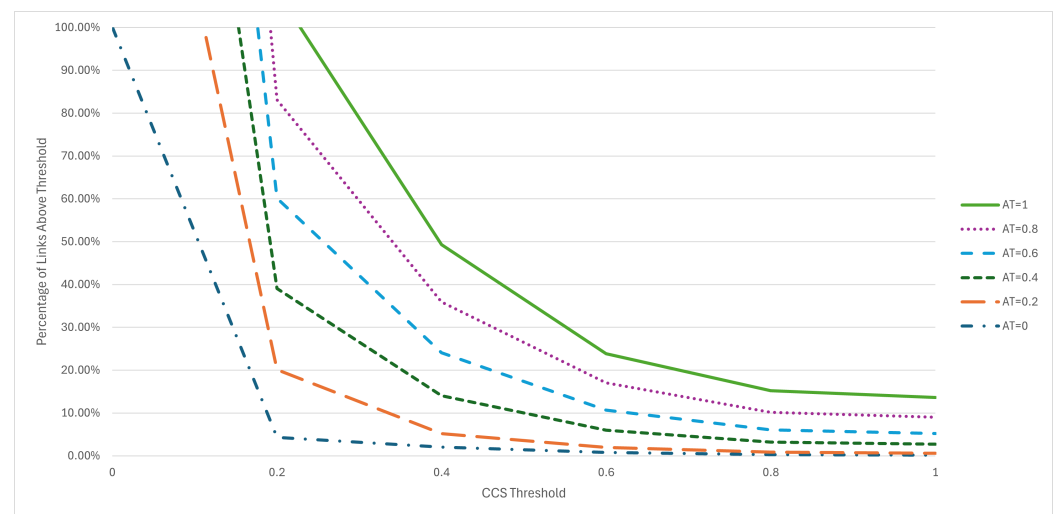


Figure 3. Graph showing percentage of links above CCS threshold in the entire dataset for activity threshold and CCS threshold values of 0 to 1.

4.4. Variation in Average Co-Committal Strength with CCS Threshold

Figure 4 shows that the mean, mode and median CCS per link increased as the CCS threshold increased, as expected. However, the behavior of the average CCS per link under changes in the activity threshold was significantly less predictable. While it was observed that the average CCS per link of Repositories 1 and 3 generally increased with activity threshold regardless of CCS threshold value, the average CCS of most other repositories could be seen to generate lower mean CCS results under certain conditions in the CCS and activity thresholds. These conditions generally included moderate values, between 40% and 60% in both thresholds, and usually had little to no correlation with the average CCF per link. In some cases, it was seen that the same changes in activity threshold that decreased the average CCS per link also increased the average CCS per link. This reinforces the view that co-committal strength and co-committal frequency are related but distinct metrics, each capturing different aspects of change coupling between two files. This distinction also shows that CCS and CCF should be interpreted as complementary proxies capturing different dimensions of evolutionary coupling, and that evolutionary coupling cannot be inferred from co-committal intensity or frequency alone, but benefits from considering both dimensions jointly.

Furthermore, the mean CCS value for Repository 5 peaks at activity thresholds of 0%. Repository 5 is sensitive to activity threshold values, with the top contributor having a relatively low percentage of overall contribution. Therefore, it can be said that the more

evenly distributed the number of commits between authors, the more likely the standard deviation of CCS per link is to decrease with activity threshold.

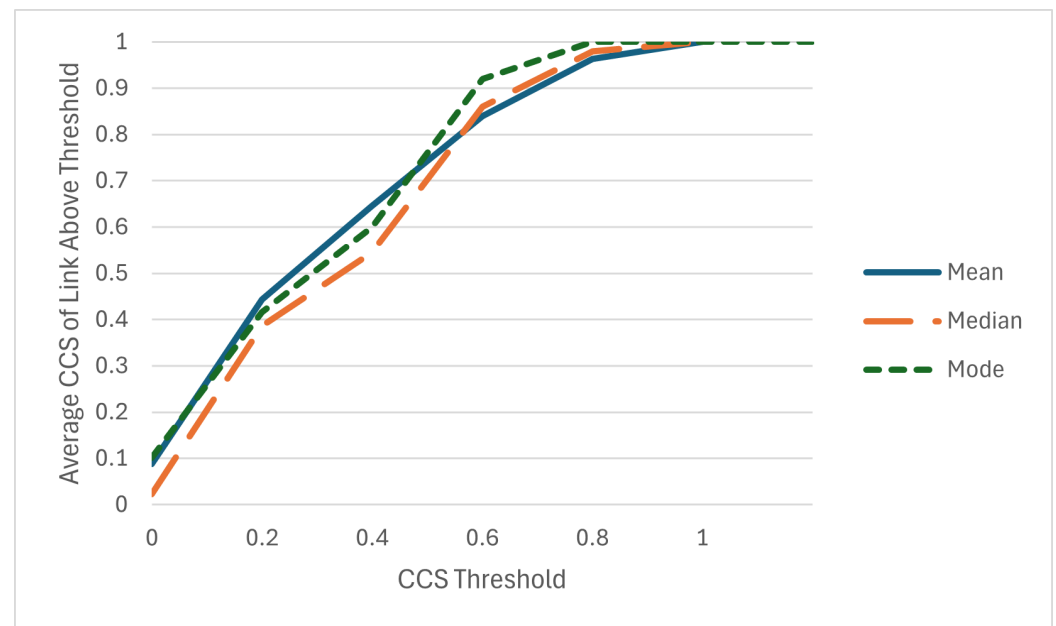


Figure 4. Average CCS of a file link above threshold over CCS threshold, for CCS threshold values from 0 to 1 and a fixed activity threshold value of 0.

4.5. Relationship Between Average Co-Committal Strength and Frequency

As shown in Figure 5, the relationship between co-committal strength (CCS) and co-committal frequency (CCF) is non-linear and varies across threshold settings. The modal and median CCF values per link are only minimally affected by changes in both activity and CCS thresholds, indicating that the typical co-committal frequency observed for most file pairs remains relatively stable under thresholding. In contrast, the mean and standard deviation of CCF exhibit pronounced peaks at moderate CCS threshold values, most notably between 40% and 60%.

This pattern indicates that file pairs exhibiting moderate co-committal strength tend to be co-committed more frequently over time than those with either very low or very high CCS values. In other words, moderately coupled file pairs are more prevalent and repeatedly co-evolve across a larger number of commits, leading to higher average CCF values. Conversely, file pairs with very high CCS values—while strongly coupled when they do co-change—appear less frequently overall, resulting in lower average CCF values at higher CCS thresholds.

These results demonstrate that CCS and CCF capture complementary but distinct dimensions of co-change behavior. High CCS does not imply high CCF, and strong co-committal relationships are not necessarily the most persistent over time. Instead, evolutionary coupling is more consistently reflected by file pairs that exhibit sustained co-changes in moderate strength across many commits. This observation reinforces prior findings that the frequency of co-changes between two files does not exhibit a direct positive relationship with the similarity of their change sets [19]. Taken together, these results highlight the importance of considering both CCS and CCF jointly when assessing evolutionary coupling, as relying on either metric in isolation may lead to incomplete or misleading conclusions.

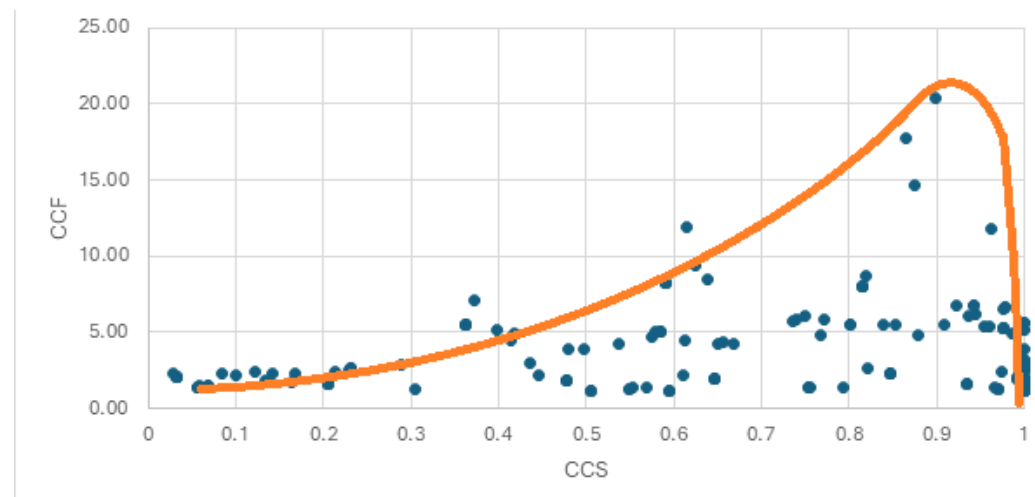


Figure 5. Comparing the average CCF to average CCS across all repositories for various threshold permutations, with trendline.

4.6. Variability in Co-Committal Frequency Across Activity Thresholds

As seen in Figure 6, the standard deviation of the mean CCF has a tendency to decrease when the activity threshold increases. This indicates that when an author makes a significant amount of commits, they are less likely to make commits in general that have evolutionary coupling as their behavior does not deviate as much from the mean as lower-activity authors. Inversely, this means that low and moderate-activity authors have a higher variance in the mean CCF of file links they created, and therefore may be more likely to create software artifacts which exhibit evolutionary coupling.

This combination of higher mean co-committal frequency and increased variability at lower activity thresholds suggests that lower-activity authors tend to introduce co-committal relationships in a more episodic manner. Their contributions are characterized by occasional bursts of activity involving multiple files, rather than steady, incremental co-evolution over time. In contrast, higher-activity authors exhibit more stable and evenly distributed co-committal behavior across commits.

This behavioral pattern also suggests practical implications for how co-committal information should be interpreted in applied settings. The combination of higher mean co-committal frequency and higher variability at lower activity thresholds indicates that co-committal links introduced by lower-activity contributors are more episodic and uneven over time. From a project management perspective, such bursty co-committal behavior may correspond to onboarding activities, occasional feature contributions, or maintenance tasks, rather than sustained responsibility for tightly coupled components.

For change impact analysis tools, this implies that co-committal relationships should not be interpreted uniformly across all contributors. Co-committal links originating from lower-activity developers may reflect short-lived or situational changes, whereas links associated with higher-activity developers are more likely to represent stable co-evolutionary relationships. Incorporating developer activity or experience levels into co-committal-based analyses may therefore help distinguish transient coupling effects from persistent evolutionary coupling.

Another interesting observation is that the aggregated standard deviation of average CCF peaks at the same CCS threshold value as the mean CCF itself. This implies that datasets with high CCF tend to also have high variance in the result set, and therefore that developers whose work generally contains significant levels of evolutionary coupling are likely to vary between levels of said coupling.

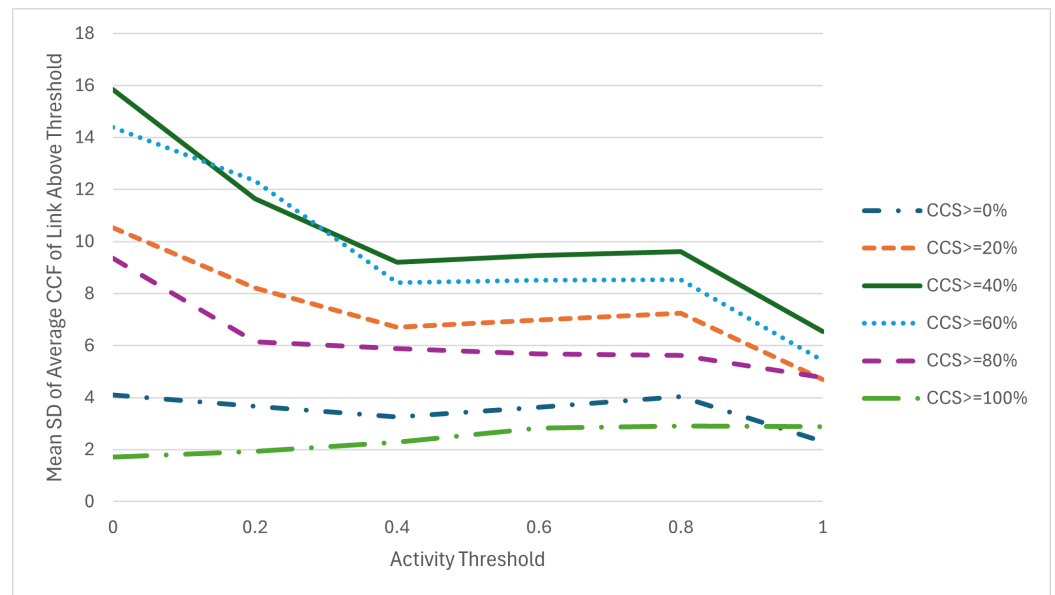


Figure 6. Graph of mean standard deviation of average CCF of a link above threshold over the activity threshold, for CCS threshold values from 0 to 1.

4.7. Effect of Activity Threshold on Mean Co-Committal Strength

As shown in Figure 7, the effect of increasing the activity threshold on the mean CCS depends strongly on the development characteristics of each repository rather than following a single consistent trend. While the average CCS of a link above the CCS threshold increases monotonically as the CCS threshold itself is raised, its behavior under changes in the activity threshold is less uniform and varies across repositories.

For two of the five repositories, increasing the activity threshold at a fixed CCS threshold of 40% leads to a decrease in the mean CCS of the remaining links. This indicates that filtering the dataset to include only more active developers does not necessarily retain the strongest co-committal relationships. Instead, these highly active developers often introduce additional co-committal links that satisfy the threshold but lie closer to its lower bound, thereby reducing the overall average CCS. In contrast, other repositories exhibit stable or increasing mean CCS values as the activity threshold increases, suggesting a tighter concentration of strong co-committal relationships among their more active contributors.

This divergence highlights that developer activity alone is not a reliable predictor of stronger co-committal behavior. Rather, the interaction between activity thresholding and co-committal strength reflects repository-specific development practices, such as differences in task decomposition, commit granularity, or collaboration patterns. Consequently, activity-based filtering can have competing effects: it may reduce noise by excluding sporadic contributors, while simultaneously introducing a broader range of moderate-strength co-committal links from highly active developers. These results underscore the importance of interpreting mean CCS values in conjunction with repository context and threshold settings, rather than assuming a uniform strengthening effect from increased activity filtering.

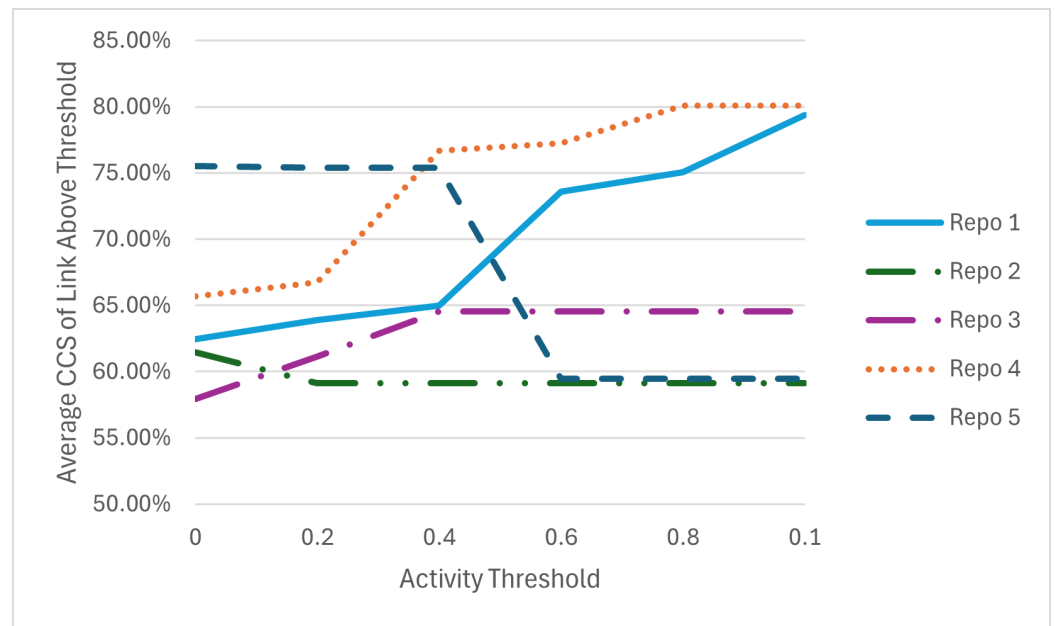


Figure 7. Graph showing the mean CCS of file links above CCS threshold over activity threshold for all repositories, for a fixed CCS threshold value of 40% and activity threshold values from 0 to 1.

4.8. Stability and Central Tendency of Co-Committal Strength Under Thresholding

Table 2 summarizes how the variability in average co-committal strength per link changes under different combinations of activity and CCS thresholds. Across all activity thresholds, the standard deviation of average CCS per link peaks at a CCS threshold of 0.2 and then decreases markedly as the CCS threshold increases. This behavior is partly structural: as the CCS threshold approaches higher values, the remaining links increasingly cluster near the upper bound of the metric, which naturally reduces variation and drives the standard deviation toward zero.

The interaction with the activity threshold further reveals how developer filtering affects CCS stability. At lower CCS thresholds, increasing the activity threshold generally leads to higher and more fluctuating standard deviation values. This reflects the inclusion of a broader range of co-committal relationships introduced by highly active developers, which increases heterogeneity in co-committal strength among the retained links. In contrast, at higher CCS thresholds, increasing the activity threshold consistently reduces the standard deviation. Under these stricter conditions, both the number of retained links and the number of contributing developers are reduced, resulting in a smaller and more homogeneous set of strong co-committal relationships.

Complementing these observations, Table 3 shows that the median CCS per link generally increases with the activity threshold across most CCS threshold values. This indicates that links associated with higher-activity developers tend, on average, to exhibit stronger co-committal strength once weak relationships are filtered out. Notably, at higher CCS thresholds, the median rapidly approaches 100%, suggesting that the remaining links represent consistently strong co-change relationships rather than marginal cases near the threshold.

When combined, these results highlight a trade-off between stability and coverage in CCS-based analysis. Lower CCS thresholds preserve a larger and more variable set of links, while higher thresholds yield a smaller but more stable and homogeneous subset of strongly co-committed file pairs. Activity-based filtering amplifies this effect by further concentrating the analysis on developers whose contributions produce consistently strong co-committal relationships. These findings underscore the importance of jointly considering

CCS and activity thresholds when interpreting co-committal strength, as threshold choices directly influence both the stability of the metric and the scope of the analysis.

Table 2. Mean standard deviation of the average co-committal strength (CCS) per file link above the specified CCS threshold, aggregated across all five repositories. Rows correspond to activity threshold (AT) values from 0 to 1, and columns correspond to CCS threshold values from 0 to 0.8. Each cell reports the mean of the per-repository standard deviation values.

	CCS = 0	CCS = 0.2	CCS = 0.4	CCS = 0.6	CCS = 0.8
AT = 0	16.27%	22.89%	20.11%	14.69%	6.60%
AT = 0.2	18.37%	23.79%	21.00%	15.36%	4.91%
AT = 0.4	20.95%	25.18%	21.59%	15.21%	3.82%
AT = 0.6	21.69%	24.93%	21.53%	13.57%	3.64%
AT = 0.8	22.51%	25.05%	21.47%	13.12%	3.46%
AT = 1	22.54%	25.43%	21.49%	12.64%	3.41%

Additionally, the mode and median CCS per link (see Table 3) generally tended to increase alongside the activity threshold, which enforces the implication that co-committal change sets created by higher-contributing authors are on average more strongly linked.

Table 3. Aggregated median co-committal strength (CCS) per file link above the specified CCS threshold, computed across all five repositories. Rows correspond to activity threshold (AT) values from 0 to 1, and columns correspond to CCS threshold values from 0 to 0.8. Values represent the mean of repository-level medians. For a CCS threshold of 1.0, all retained links have a CCS value of 100% by definition.

	CCS = 0	CCS = 0.2	CCS = 0.4	CCS = 0.6	CCS = 0.8
AT = 0	2.27%	38.58%	54.44%	86%	98%
AT = 0.2	3.05%	40.67%	87.29%	100%	100%
AT = 0.4	4.46%	42.67%	95.15%	100%	100%
AT = 0.6	5.55%	46%	60%	95.15%	100%
AT = 0.8	6.87%	46%	63.41%	95.15%	100%
AT = 1	7.35%	46%	70%	95.15%	100%

4.9. Developer Commit Activity Under Increasing Activity Thresholds

Figure 8 shows that the mean number of commits per author increases as the activity threshold increases, as expected. Similarly, the number of authors tends towards 1 as the activity threshold increases, meaning that the mean number of commits for activity threshold values of 100% is the number of commits made by the most active author in the selected repository.

Additionally, the mode and median of the number of commits per author (see Table 4) were positively correlated with the value of the activity threshold, which was also the expected outcome for these metrics.

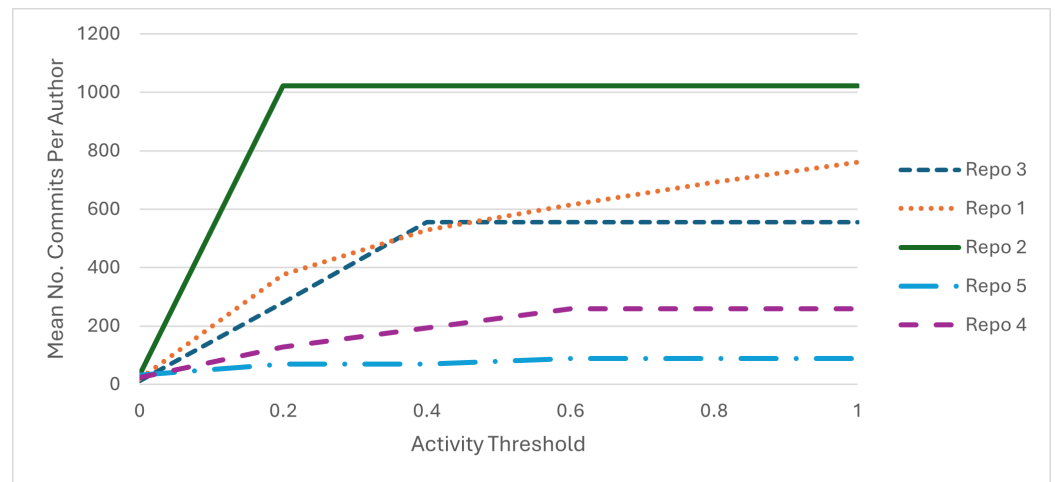


Figure 8. Graph showing the mean of the number of commits per author for activity threshold values of 0 to 1 across all repositories.

Table 4. Aggregated mode and median number of commits per author for different activity threshold (AT) values from 0 to 1, computed across all five repositories. The table summarizes how author-level commit activity changes as increasingly active contributors are selected.

	Median	Mode
AT = 0	2.3	1.6
AT = 20	308.5	67.4
AT = 40	422.4	182.4
AT = 60	459.4	220.4
AT = 80	471.8	252
AT = 100	485	280.6

4.10. Variability in Developer Commit Activity Across Activity Thresholds

The standard deviation of the average number of commits per author increases with low non-zero activity threshold values, then peaks and starts decreasing as the threshold reaches 100%. This suggests that there is more variance from the mean after the lowest-contributing authors are removed from the sample. This is enforced by the observation that the mode and number of commits per author are usually a low value, and more often than not 1, for activity threshold values of 0%. Lower activity threshold values generally have lower mean CCS per link than their higher-threshold counterparts, but also higher CCF. This implies that less active authors are more likely to make commits, the files of which display evolutionary coupling.

4.11. Developer Commit Rate Across Activity Thresholds

The mean number of commits per day is dependent on the development style of the authors within the repositories, with some repositories (e.g., 3) having an increasing or very similar (e.g., 2) number of commits per day between each activity threshold level, but most repositories having fewer commits per day as the activity threshold increased as seen in Figure 9 with the standard deviation in Figure 10. This suggests that the more commits an author makes, the more “tangled” these commits are. This would be a factor in why average CCS tends to increase with activity threshold, in addition to having significant implications for change impact analysis. This is because means commits by more active authors may be, on average, more likely to be miscategorized as bug fixes [11]. This implication is further

confirmed by the observation that the percentage of links above threshold increases with the activity threshold, as can be seen in Figure 3.

Across all repositories analyzed, the modal and mean number of commits remained constant at 1 regardless of activity threshold values. This shows that across all development styles exhibited by the dataset, developers tend to commit daily, which could also indicate tangled commits, if not for the mode and median number of files changed per commit remaining relatively small compared to the mean across the majority of repositories evaluated. If these values were larger, it would be a clearer indicator of a high level of evolutionary coupling.

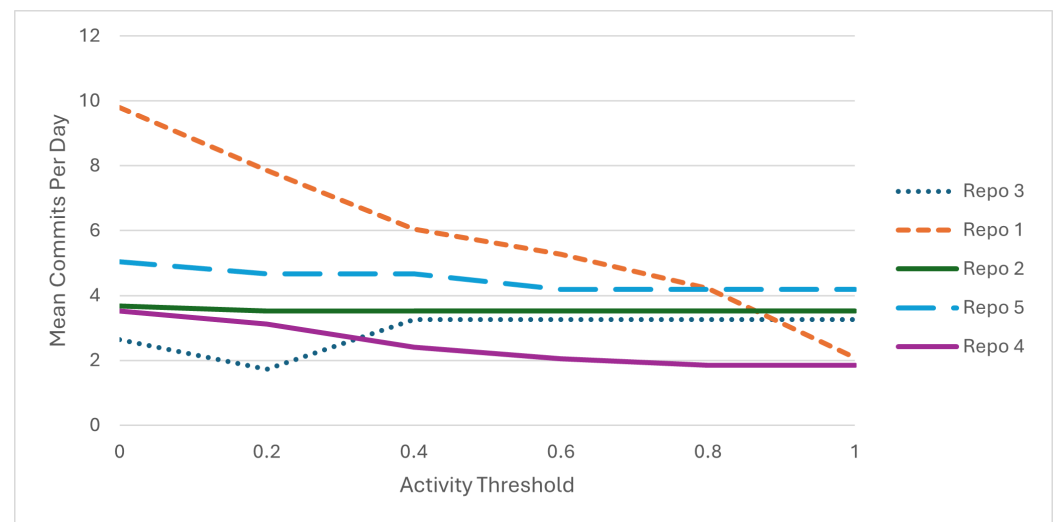


Figure 9. Graph showing the mean number of commits per day for activity threshold values of 0 to 1.

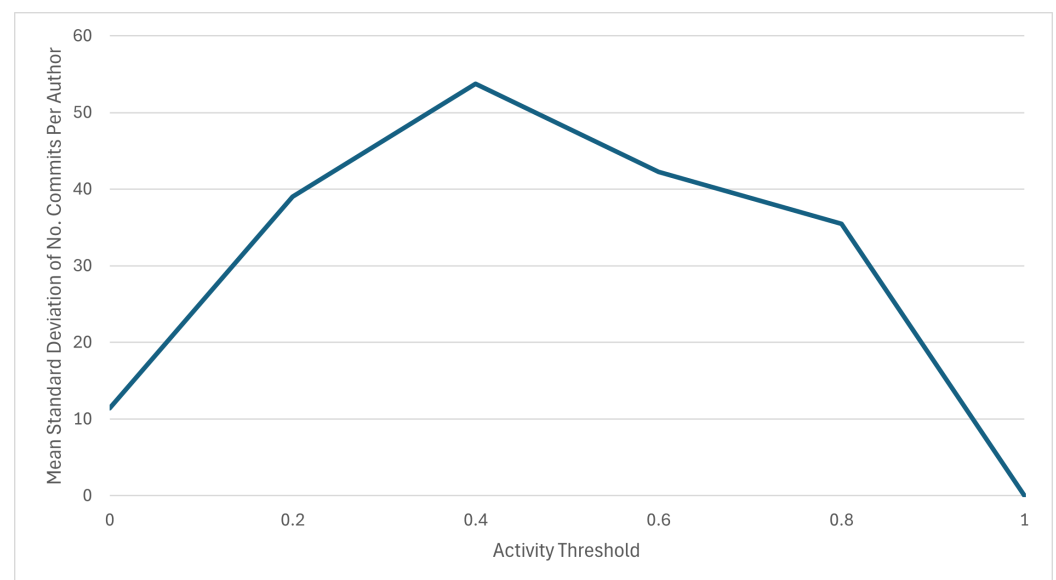


Figure 10. Graph showing the mean of the standard deviation of the number of commits per author for activity threshold values of 0 to 1 across all repositories.

4.12. Commit Granularity and Temporal Variability Under Activity Thresholding

The standard deviation of the number of commits per day is dependent on the development style (Figure 11). In general, this value is relatively low, ranging between 0.05 and 0.27. This indicates that most developers, regardless of activity level, personally tend to make the same number of commits per day relative to their previous behavior.

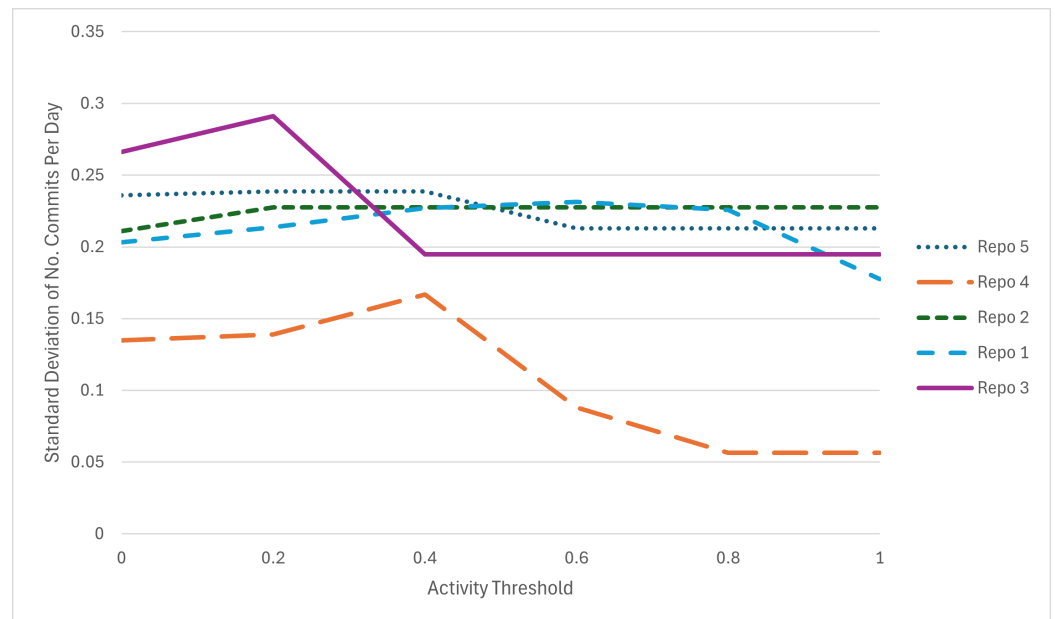


Figure 11. Graph showing the standard deviation of the number of commits per day for activity threshold values of 0% to 100%.

Depending on the repository, the number of file changes per commit can increase, decrease or fluctuate with increased activity threshold (Figure 12). This is reflective of the varying development styles of these repositories—the higher contributing authors could either have changed a specific amount of files per commit for a low standard deviation, or a varied number of files per commit for a high standard deviation. For instance, Repository 3's mean file changes per commit increased from 37.5 at an activity threshold of 0 to 82.9 when limited to the highest-contributing author. The standard deviation also increased at the latter level. This indicates that for this repository, authors who make more commits tend to vary more in terms of how many files are changed in each commit, while lower-contributing authors are more consistent.

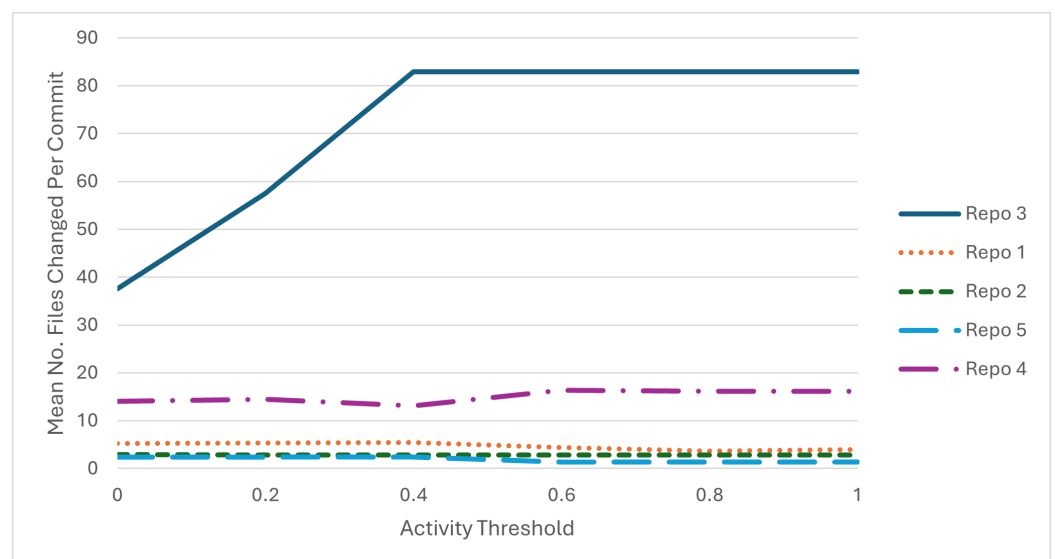


Figure 12. Graph showing the mean number of files changed per commit for activity threshold values from 0 to 1 for all repositories.

4.13. Joint Behavior of Co-Committal Strength and Frequency Across Activity Thresholds

This is also shown by the observation in Figure 13 that lower-contributing authors tend to have lower mean CCF, lower mean CCS and lower standard deviation for both metrics. In addition, moderately active authors (generally in activity ranges between 0.4 and 0.8) tend to make commits with high mean CCF, moderate mean CCS and high standard deviations for both metrics. This indicates that, despite differing development styles, authors who fall within a median percentile level of activity generally tend to also have the most evolutionary coupling within their commits.

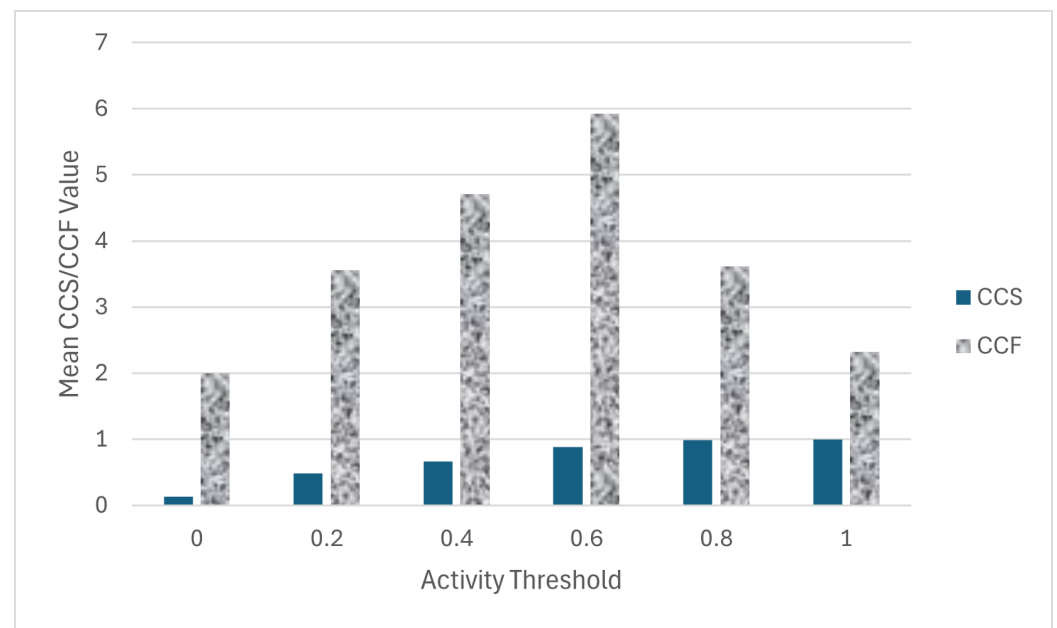


Figure 13. Graph showing the mean CCS and CCF of file links for activity threshold values from 0 to 1 for all repositories.

An additional example of how the mean number of files changed per commit varies across repositories can be seen in the behavior of Repository 1 in Figure 12. For this repository, the number of files changed decreased as the activity threshold increased. It could be the case, given these results, that a more even spread in the number of commits across authors does not necessarily indicate that the amount of commit content will be consistent.

Overall, the behavior of the mean CCF over the activity threshold in this dataset is comparable to the results found by [6], where it was found that change set sizes, across various different repositories as part of a large-scale analysis, had a tendency to peak to peak at moderate activity threshold values and then decrease. The findings in the dataset utilized in this analysis corroborate their observed behavior on a smaller scale, indicating that this is a trait that may be inherent to common development styles.

4.14. Variability in Commit Scope Across Activity Thresholds

It can be observed in Figure 14 that the standard deviation of the average number of files changed per commit was highly variable, both between repositories and internally based on changes in the activity threshold. In 2 out of 5 cases, increasing the activity threshold decreased the standard deviation of the number of files changed per commit, indicating that in this case, more active authors made a more consistent number of changes across their set of commits. For other cases, this metric displayed significantly different behavior.

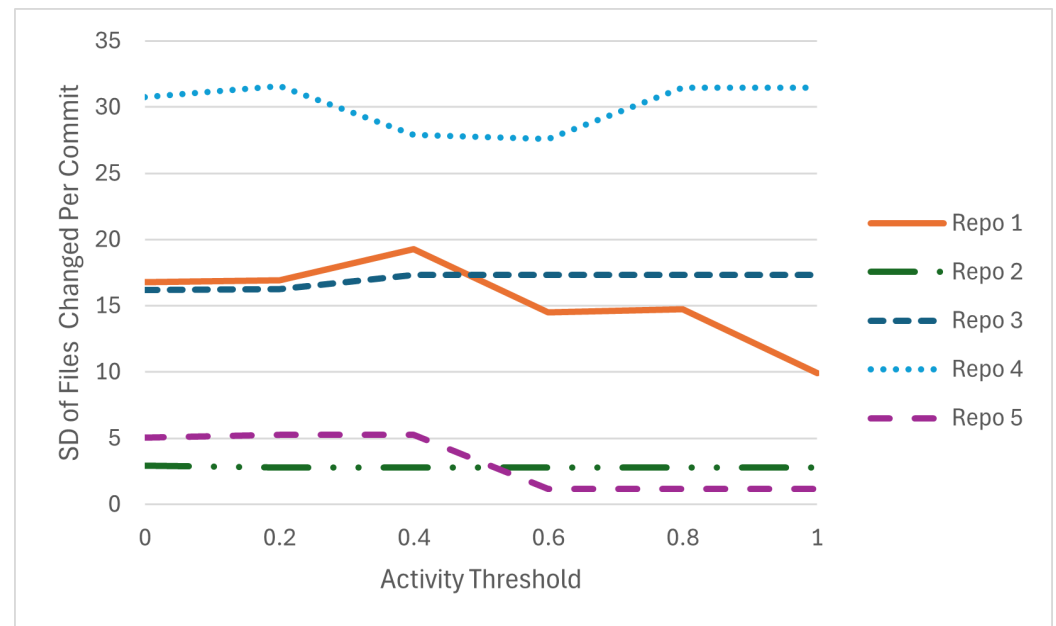


Figure 14. Standard deviation of average number of files changed per commit for activity threshold values from 0 to 1 for all 5 repositories.

For Repository 2, the standard deviation of the number of files changed per commit remained constant. Given that in Figure 12 it can be observed for this repository that the mean also exhibits little change for authors of different activity levels, this indicates that the average commit content and the variance from it were unaffected by changes in the activity threshold for this repository.

For Repository 3, there was a minor increase in the standard deviation as the activity threshold increased, implying that the more active authors for this repository tended to vary more in the content of their commits.

The most noteworthy behavior in this metric comes from Repository 4, where the standard deviation is lowest for moderately active authors and higher for both low- and high-activity authors. This implies that the mean number of files changed per commit across the set of commits by multiple lower-activity authors has a comparable standard deviation to that of the same metric for the set of commits by a smaller number of high-activity authors, while more moderately active authors are more consistent in their behavior, as well as higher in the overall level of evolutionary coupling displayed by their commits.

Furthermore, the standard deviation of the mean number of files changed per commit also exceeds the mean in all cases for Repository 4. This implies that for most repositories, the number of files changed per commit has a significant amount of spread from the mean, regardless of activity level.

4.15. Distributional Characteristics of Commit Scope Across Activity Thresholds

The difference between the standard deviation and the mean for the average number of files changed per commit can be seen in Figure 15, in which four out of the five repositories have a larger standard deviation than the mean (i.e., the difference between the standard deviation and the mean is positive). This implies not only a significant variation in the number of files changed per commit but also that this variation is one-sided.

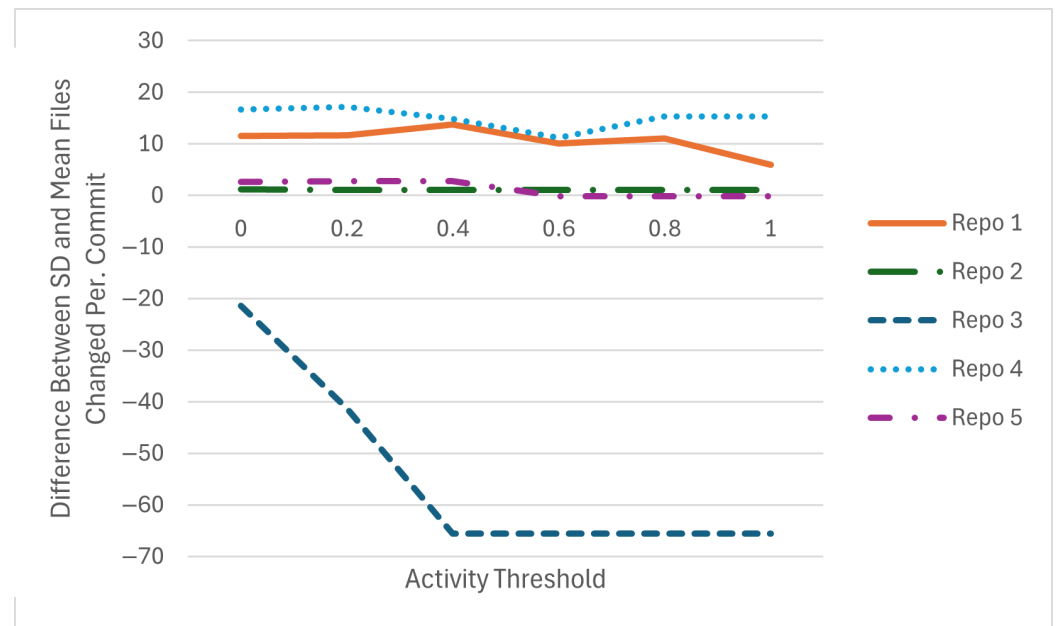


Figure 15. Graph showing the difference between the standard deviation and the mean for the average number of files changed per commit, for activity threshold values from 0% to 100% for all 5 repositories.

Comparing this to the mode (visible in Table 5) and median number of files changed per commit for these repositories, which in the majority of cases are both 1, this implies that there are significantly more values below the mean than above it for most repositories due to most commits being small and containing minimal amounts of useful data for co-committal link analysis.

Table 5. Median of the number of files changed per commit for activity threshold values from 0 to 1 for all repositories.

	1	2	3	4	5
AT = 0	1	2	1	6	1
AT = 0.2	1	2	1	6	1
AT = 0.4	1	2	1	6	1
AT = 0.6	1	2	1	9	1
AT = 0.8	1	2	1	9	1
AT = 1	1	2	1	9	1

It is also observable in Figure 15 that the only repository where the mean was consistently larger than the standard deviation is Repository 4, indicating that its file change sets have the smallest, and most symmetrical, variance from the mean in the dataset. Other traits exhibited by Repository 4 include a high sensitivity to the activity threshold due to a more even distribution of the number of commits between authors, and the lowest number of authors in the dataset at 41. This indicates that co-committal patterns vary significantly more in repositories with large numbers of authors.

Repository 4 also had a relatively low number of commits per day and the second highest number of files changed per commit for any of the five repositories evaluated. This may indicate that for this repository, a long software evolution lifecycle with multiple tangled commits [22] have contributed towards the level of evolutionary coupling presented by its software artifacts.

These results also provide important context for interpreting co-committal behavior. Commits that modify many files at once can introduce co-committal links as a consequence of commit structure rather than sustained evolutionary coupling between files. This occurs because co-committal analysis operates at the granularity of commits, treating all files modified together as related regardless of whether the changes reflect a shared long-term dependency. As a result, repositories or activity-threshold settings exhibiting larger commit scopes are more susceptible to inflated co-committal measures due to tangled commits [22], which can mediate the observed strength and frequency of co-committal relationships.

4.16. Threats to Validity

The results of this analysis are not without potential threats to their validity.

4.16.1. Internal Validity

A potential threat to internal validity stems from the reliance on co-committal information extracted from version-control commits as a proxy for evolutionary coupling. Co-committal relationships may be influenced by commit practices such as tangled commits, where multiple unrelated changes are bundled into a single commit, thereby introducing file links that do not reflect sustained co-evolution. Although the analysis incorporates activity-based and CCS-based thresholding to mitigate incidental co-changes, some degree of noise may remain due to differences in commit granularity and developer practices across repositories. As a result, observed co-committal relationships should be interpreted as indicative rather than definitive evidence of evolutionary coupling.

A further internal limitation is that the study does not classify commits by type (e.g., bug fixes, refactorings, feature additions) and does not apply automated detection of tangled commits. As a result, any interpretations about whether high-activity developers produce more tangled or maintenance-heavy commits remain speculative. Without commit-message classification or semantic analysis of the underlying code changes, the study cannot empirically determine whether commit structure differs systematically between high- and low-activity authors. This limits the strength of conclusions regarding the relationship between developer activity and commit complexity.

A further internal limitation is that the study does not directly validate the detected co-committal links against an independent ground truth, such as known architectural dependencies, module-level relationships, or semantically verified code links. Co-committal patterns may suggest potential coupling, but without comparison to an established dependency model, the extent to which these links correspond to actual structural or semantic connections remains uncertain. This limitation reflects a constraint of commit-based MSR analysis, which relies on developer behavior rather than explicit architectural specifications.

Several steps were taken to reduce noise and minimize the influence of incidental co-changes on our results. First, the analysis employs both activity-based and CCS-based thresholding, which naturally filters out many low-signal co-committal events, including those introduced by sporadic contributors or weak file relationships. These thresholds also reduce the impact of large, infrequent, or irregular commits by limiting the dataset to authors and links that exhibit consistent co-change patterns. In addition, our analysis is performed separately for each repository before aggregated results are reported, ensuring that no single project or anomalous commit pattern disproportionately shapes the overall trends. Despite these mitigation steps, several sources of noise remain unaddressed. The study does not exclude vendor-generated files, auto-generated artifacts, or formatting-only commits, all of which can introduce co-change signals unrelated to true evolutionary coupling. Likewise, merge commits and branching structure were left in their raw Git log form and not normalized or filtered, even though they may inflate or distort co-committal

relationships by bundling unrelated changes. We also do not classify commits by intent (e.g., bug fix, refactoring, feature addition), leaving some interpretations about commit structure and author behavior necessarily speculative. These limitations highlight opportunities for future refinement of the analysis. Future work could incorporate automated detection of vendor or machine-generated files, identification of formatting-only commits using commit-message or diff analysis, and normalization of merge commits to distinguish genuine co-evolution from workflow-driven co-change. Adding commit-type classification or semantic analysis of code modifications would further allow researchers to test whether specific patterns—such as tangled commits or maintenance-heavy changes—are more prevalent among high-activity authors. Such noise-reduction enhancements would reduce threats to validity and strengthen the reliability of co-committal metrics as indicators of evolutionary coupling.

4.16.2. Construct Validity

A threat to construct validity arises from the fact that the analysis relies exclusively on version-control commit metadata to infer co-committal relationships. While co-committal provides an observable and scalable proxy for change coupling, it does not directly validate whether linked files are semantically or logically dependent at the code or architectural level. As a result, some observed co-committal relationships may reflect coincidental or process-driven co-changes rather than true semantic coupling. Consequently, the findings should be interpreted as characterizing coupling as manifested through commit behavior, rather than as a validated representation of underlying semantic dependencies. Furthermore, differences in development style across repositories are interpreted only through observable commit behavior, as organizational culture and team practices cannot be reliably inferred without qualitative data. As a result, the study cannot confirm whether the observed co-committal links correspond to true design-level or architectural relationships.

4.16.3. External Validity

A threat to external validity arises from the fact that all analyzed repositories were actively evolving at the time the data snapshot was taken (March 2024). Co-committal patterns derived from a fixed snapshot may not fully represent long-term evolutionary coupling, particularly in fast-evolving projects where architectural structure, development focus, or team composition continue to change. As development progresses, some observed co-committal relationships may strengthen through repeated co-changes, while others may weaken or disappear as features stabilize or responsibilities are refactored. Consequently, the results characterize evolutionary coupling within the observed time window and should not be assumed to represent stable coupling relationships over the entire lifespan of the projects.

A further threat to external validity is the distribution of commit volume over the five repositories included, such that Repository 1 contains some 60% of the overall commits, which may skew any generalized results disproportionately. This is mitigated as results are presented for each repository in the Results section and the general purpose of this work is to introduce the method and show potential rather than making clear statistical conclusions.

4.16.4. Conclusion Validity

A limitation of this study is that comparisons between different groups (e.g., high- and low-activity authors) are based on descriptive statistics rather than formal statistical hypothesis testing. While the large number of commits analyzed and the consistency of observed trends across repositories increase confidence in the findings, the results should be interpreted as indicative rather than statistically conclusive.

5. Conclusions and Future Work

To conclude, once the existing scientific literature on co-committal as an indicator of evolutionary coupling was evaluated, we designed and conducted the case study reported in this paper to gather more data and contribute further empirical findings in this area. Five large GitHub repositories with a combined 14,008 commits were analyzed in this study, and it was found that while development style had a major impact on the results, some metrics remained consistent. It was found that, generally, the highest and lowest CCS links both had relatively low CCF scores, meaning that co-committal strength is a more nuanced indicator of evolutionary coupling than co-committal frequency. The file links with the highest CCF tended to have a moderate CCS value, which implied that files which have been co-committed on a statistically significant proportion of their overall commits could indicate evolutionary coupling.

Additionally, the behavior, in terms of commit patterns, of authors who made more commits was found to be markedly different from those who made fewer commits. Evidence was found to suggest that authors who make more commits tend to have more files per commit, which is an indicator of the “tangled commits” issue discussed in the literature [22]. In such cases, commits that modify many files at once can artificially increase observed co-committal links, even when the files are not consistently evolving together over time. This arises because co-committal analysis operates at the granularity of commits, so commits that bundle multiple unrelated changes generate co-committal links that do not reflect long-term evolutionary coupling.

The study also found that the results varied based on the level of activity of the authors in the dataset, as the average CCS of a link by a higher-contributing author was generally higher than that of an author who contributed a small number of commits to the same repository. Across all repositories, it was usually the case that lower contributing authors had the highest mean CCF per link, but also the highest standard deviation from the mean of this metric. It was also determined from the dataset that repositories that were more sensitive to changes in the activity threshold, i.e., a more even distribution of commits between authors, tended to have a higher level of evolutionary coupling.

The results also indicate that co-committal is a useful indicator of evolutionary coupling only under specific and observable development conditions. In particular, co-committal-based analysis is most informative when: (1) development activity is sustained rather than sporadic, with contributors exhibiting moderate and consistent levels of activity over time; (2) commits are relatively evenly distributed, avoiding highly bursty or episodic change patterns; (3) commit scope is limited, with changes typically involving a small and stable set of files rather than large, tangled commits; and (4) co-committal strength and frequency are considered jointly, as neither metric alone reliably captures persistent co-evolution. Outside of these conditions, co-committal relationships are more likely to reflect incidental or short-lived co-changes rather than meaningful evolutionary coupling.

To make these implications more explicit, our findings suggest several concrete conditions that should be present in a repository for co-committal to serve as a reliable indicator of evolutionary coupling. First, project activity should be sustained across a reasonable number of contributors rather than dominated by a single highly active developer; repositories with a more balanced distribution of commits across authors tend to produce clearer and more stable co-committal patterns. Second, commits should typically involve a relatively small and stable number of files, as large or formatting-only commits can introduce noise that obscures genuine co-evolution between artifacts. Third, commit frequency should be moderate and consistent over time, since highly sporadic or bursty activity can create incidental co-changes that do not reflect long-term coupling. From a practical perspective, developers and project maintainers can improve the usefulness of co-committal analysis by

adopting practices that reduce noise and promote clearer change signals. These include avoiding large, multi-purpose commits, limiting formatting or automated changes to dedicated commits, and ensuring that feature- or bug-related changes are committed in small, logically cohesive units. Projects with multiple active contributors can also benefit from more consistent commit granularity and clearer division of responsibilities, both of which strengthen the interpretability of co-committal relationships. When these conditions are met, co-committal analysis becomes more actionable and can more reliably support tasks such as change-impact analysis, architectural understanding, and maintenance planning.

The findings of this study also suggest that richer historical development data could enhance software change impact analysis beyond what can be inferred from co-committal relationships alone. While co-committal captures observable co-change behavior, it does not preserve information about the broader historical context in which changes occur, such as influence chains, refactorings, or conceptual evolution over time. Prior work has shown that recording and structuring code histories can provide deeper insight into how changes propagate and influence future development decisions [23]. Our results, particularly the observed limitations of co-committal under bursty activity and tangled commits, reinforce the potential value of incorporating more expressive historical representations when assessing change impact.

In terms of future work directions, it is hoped that the findings and observations in this paper will serve to stimulate further research and development in this area. One prospective future development could be a semantic link tool to cross-reference against the file links identified in this article, e.g., a tool to analyze semantic links between files and co-committal links, then identify the intersection between them to determine if co-committal is truly an indicator of file links for use in change impact analysis. Such a tool would be able to use the data acquired in this analysis to reinforce change set predictions, and potentially also have accuracy metrics to allow for analysis of its performance. Another set of future developments could be effect normalization and weighting depending on the number of commits, in order to more accurately compare the behaviors and development styles seen in repositories of various sizes and ages.

A key direction for future work is to validate co-committal links against a ground-truth representation of system structure. This could be achieved by integrating semantic or architectural dependency extraction tools with the co-committal analysis, and identifying the overlap between semantic links and commit-derived links. Such a validation step would allow the study to assess whether co-committed file pairs reflect true architectural or logical relationships. While this idea was outlined conceptually in the present manuscript, implementing such validation—either through static dependency analysis, architectural recovery techniques, or fine-grained semantic linkage—is a necessary next step to strengthen the conclusions about whether co-committal behaviour reflects actual coupling in the codebase.

Another complementary direction for future work is the incorporation of commit-message classification or automated change-type detection. Such techniques—using natural language processing on commit messages or mining fine-grained code edits—would make it possible to distinguish between bug fixes, refactorings, feature work, and tangled commits. Integrating this additional layer of commit semantics would allow for a direct test of whether high-activity authors tend to produce more maintenance-heavy or structurally tangled commits. This enhancement would strengthen the empirical basis of the observed patterns and reduce the reliance on behavioural inference.

Author Contributions: N.P. and D.C. conceptualized the work; N.P. undertook the core research and analysis; V.G. refined the research contribution and framing. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: All source data is publicly available in the linked repositories in the paper.

Conflicts of Interest: The authors declare no conflicts of interest.

References

- Li, B.; Sun, X.; Leung, H.; Zhang, S. A survey of code-based change impact analysis techniques. *Softw. Test. Verif. Reliab.* **2013**, *23*, 613–646. [CrossRef]
- Kretsou, M.; Arvanitou, E.M.; Ampatzoglou, A.; Deligiannis, I.; Gerogiannis, V.C. Change impact analysis: A systematic mapping study. *J. Syst. Softw.* **2021**, *174*, 110892. [CrossRef]
- Rolfsnes, T.; Di Alesio, S.; Behjati, R.; Moonen, L.; Binkley, D.W. Generalizing the analysis of evolutionary coupling for software change impact analysis. In *2016 IEEE 23rd International Conference on Software Analysis, Evolution, and Reengineering (SANER)*; IEEE: Piscataway, NJ, USA, 2016; Volume 1, pp. 201–212.
- Ramadani, J.; Wagner, S. Are suggestions of coupled file changes interesting? In *Proceedings of the International Conference on Evaluation of Novel Software Approaches to Software Engineering, Rome, Italy, 27–28 April 2016*; SCITEPRESS: Setúbal, Portugal, 2016; Volume 2, pp. 15–26.
- Canfora, G.; Cerulo, L. Impact analysis by mining software and change request repositories. In *Proceedings of the 11th IEEE International Software Metrics Symposium (METRICS'05)*, Como, Italy, 19–22 September 2005.
- Cohen, E.; Consens, M.P. Large-scale analysis of the co-commit patterns of the active developers in github's top repositories. In *Proceedings of the 15th International Conference on Mining Software Repositories*, Gothenburg, Sweden, 28–29 May 2018.
- Kanwal, J.; Maqbool, O.; Abbasi, R.; Abbasi, A.Q. Network analysis of software change history for understanding software evolution. In *Proceedings of the 17th IEEE International Multi Topic Conference*, Karachi, Pakistan, 8–10 December 2014.
- Zimmermann, T.; Zeller, A.; Weissgerber, P.; Diehl, S. Mining version histories to guide software changes. *IEEE Trans. Softw. Eng.* **2005**, *31*, 429–445. [CrossRef]
- Hagward, A. Using Git Commit History for Change Prediction: An Empirical Study on the Predictive Potential of File-Level Logical Coupling. 2015. Available online: <https://api.semanticscholar.org/CorpusID:58954770> (accessed on 4 February 2024).
- Wiese, I.S.; Ré, R.; Steinmacher, I.; Kuroda, R.T.; Oliva, G.A.; Treude, C.; Gerosa, M.A. Using contextual information to predict co-changes. *J. Syst. Softw.* **2017**, *128*, 220–235. [CrossRef]
- Herzig, K.; Zeller, A. The impact of tangled code changes. In *Proceedings of the Working Conference on Mining Software Repositories (MSR)*, San Francisco, CA, USA, 18–19 May 2013.
- Ufuktepe, E.; Tuglular, T.; Palaniappan, K. The relation between bug fix change patterns and change impact analysis. In *Proceedings of the 2021 IEEE 21st International Conference on Software Quality, Reliability and Security (QRS)*, Haikou, China, 6–10 December 2021.
- Agrawal, A.; Singh, R.K. Predicting co-change probability in software applications using historical metadata. *IET Softw.* **2020**, *14*, 739–747. [CrossRef]
- Hanam, Q.; Mesbah, A.; Holmes, R. Aiding code change understanding with semantic change impact analysis. In *Proceedings of the 2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, Cleveland, OH, USA, 30 September–4 October 2019.
- Huang, Y.; Jiang, J.; Luo, X.; Chen, X.; Zheng, Z.; Jia, N.; Huang, G. Change-patterns mapping: A boosting way for change impact analysis. *IEEE Trans. Softw. Eng.* **2021**, *48*, 2376–2398. [CrossRef]
- Ramadani, J.; Wagner, S. Which change sets in Git repositories are related? In *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*, Vienna, Austria, 1–3 August 2016.
- Higo, Y.; Hayashi, S.; Kusumoto, S. On tracking Java methods with Git mechanisms. *J. Syst. Softw.* **2020**, *165*, 110571. [CrossRef]
- Mondal, M.; Roy, B.; Roy, C.K.; Schneider, K.A. Investigating the relationship between evolutionary coupling and software bug-proneness. In *Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering*, Toronto, ON, Canada, 4–6 November 2019.
- Saydemir, A.; Simitcioglu, M.E.; Sozer, H. On the use of evolutionary coupling for software architecture recovery. In *Proceedings of the 2021 15th Turkish National Software Engineering Symposium (UYMS)*, Izmir, Turkey, 17–19 November 2021.
- Sohn, J.; Papadakis, M. Using Evolutionary Coupling to Establish Relevance Links Between Tests and Code Units. A case study on fault localization. *arXiv* **2022**, arXiv:2203.11343. [CrossRef]
- Garrity, J.; Cutting, D. A Common Language of Software Evolution in Repositories (CLOSER). *Software* **2025**, *4*, 1. [CrossRef]

22. Pârundefinedachi, P.P.; Dash, S.K.; Allamanis, M.; Barr, E.T. Flexeme: Untangling commits using lexical flows. In Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2020, New York, NY, USA, 8–13 November 2020. [[CrossRef](#)]
23. Pham, V.T.T.; Kelleher, C. Code histories: Documenting development by recording code influences and changes in code. *J. Comput. Lang.* **2025**, *82*, 101313. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.